

None of this is true yet. This is the CV a junior AI engineer should be able to write honestly by **September 2027**, starting from today. Six projects, each one a system rather than an exercise. Work them in order, because the later ones assume the earlier ones.

AI ENGINEER

[Your Name]

you@email.com +92 3... github.com/you linkedin.com/in/you Islamabad, Pakistan

PROFESSIONAL SUMMARY

AI Engineer who builds generative, agentic, and vision systems end to end, from the training loop to the production SLO. Comfortable below the API line: implements transformer internals from scratch, runs the full fine-tuning ladder from continued pre-training through preference optimization, fine-tunes detection models on custom datasets, and serves quantized models on self-hosted inference. Ships retrieval and agent platforms with multi-tenant isolation and evaluation harnesses attached, and treats latency, cost per request, and failure modes as first-class design constraints.

TECHNICAL SKILLS

CORE DEEP LEARNING

PyTorch, NumPy, Hugging Face Transformers · transformer internals from scratch (BPE, rotary embeddings, grouped-query attention, KV cache, SwiGLU, RMSNorm) · CNNs, Vision Transformers · memory accounting and optimization: activation memory, gradient checkpointing, mixed precision, gradient accumulation, KV-cache sizing · learning-rate scheduling, distributed data parallel

POST-TRAINING

Continued pre-training, supervised fine-tuning · preference optimization with DPO · RLHF with a learned reward model, PPO, and KL control · GRPO against verifiable rewards, RL environment design, verifier authoring, reward-hacking detection and mitigation · LoRA and QLoRA · Hugging Face TRL, PEFT, Unsloth, Axolotl · catastrophic-forgetting mitigation, knowledge distillation, hyperparameter search

AGENTIC SYSTEMS

Harness engineering: the agent loop, tool dispatch, output parsing, malformed-call recovery, retry and backoff, step budgets, termination · context engineering: context budgeting, structured assembly, compaction and summarization, eviction policy, prompt-cache-aware prompt ordering, measuring degradation over long sessions · short-term working state versus durable long-term memory, retrieval keys, expiry · LangGraph, LangChain, CrewAI, Model Context Protocol · tool calling, ReAct, plan-and-execute, reflection, multi-agent routing, stateful graph execution, checkpointing, human-in-the-loop interrupts · n8n workflow automation

RETRIEVAL SYSTEMS

Chunking strategy design, hybrid BM25 and dense search, contextual retrieval, cross-encoder reranking, HyDE, query decomposition, context-window tuning · Qdrant, pgvector, ChromaDB, FAISS · multi-tenant namespace isolation, async ingestion via RabbitMQ or Celery

COMPUTER VISION & NLP

YOLO and RT-DETR fine-tuning on custom datasets · anchor calibration, IoU thresholds, non-max suppression, mosaic and HSV augmentation, mean average precision · segmentation with SAM · OCR with PaddleOCR and Tesseract, deskewing, adaptive thresholding, contour-guided crop alignment · spaCy NER with custom entity schemas, information extraction · OpenCV

SPEECH & MULTIMODAL

Whisper, voice activity detection, streaming ASR and TTS, speaker diarization, barge-in handling, endpointing · CLIP, vision-language models, document understanding

INFERENCE & SERVING

vLLM, TensorRT-LLM, Ollama, llama.cpp · paged attention, continuous batching, speculative decoding · AWQ, GPTQ, GGUF quantization · inference-memory budgeting and VRAM prediction,

concurrency and throughput curves · ONNX and TensorRT export, INT8 edge deployment

EVALUATION & SAFETY

Offline evaluation suites: deterministic replay with recorded fixtures, programmatic scoring before judged scoring, regression corpora mined from production traces, slice-level reporting, run diffing, flakiness quarantine, and quality, cost, and latency budgets enforced together as a merge gate · numerical correctness testing against reference implementations, golden-set construction, LLM-as-judge with human agreement calibration, RAGAS, pairwise preference evals · perplexity, precision, recall, F1, nDCG, mean average precision, word error rate · distributed tracing and per-request cost attribution · drift detection, prompt-injection defence, PII redaction, output guardrails, red-teaming

DATA & ANALYSIS

Training-corpus construction: sourcing, deduplication, quality filtering, eval-set contamination checks, distribution and coverage analysis, annotation guidelines and inter-annotator agreement, dataset cards · significance testing and bootstrap confidence intervals on model comparisons, sample-size estimation · SQL analytics, schema modelling, natural-language-to-SQL, exploratory analysis and dashboards over production telemetry · Pandas, Matplotlib, Grafana

ENGINEERING & PLATFORM

Python, TypeScript, SQL, Bash · FastAPI, Pydantic, SQLAlchemy, Celery, Node.js, Next.js · PostgreSQL, Redis, MongoDB · Docker, Kubernetes, Nginx, Terraform, GitHub Actions · MLflow, Weights & Biases, LangFuse, OpenTelemetry, Prometheus, Grafana · AWS, Azure

EXPERIENCE

AI Engineer

Sept 2026 – Present

[Company] · Islamabad / Remote

- Own a generative AI feature end to end, covering data collection, model selection, evaluation harness, serving infrastructure, and the production on-call rotation for it.
- Replaced an LLM hop in the request path with a fine-tuned encoder classifier, holding task accuracy within one point while cutting per-request cost and p95 latency by roughly an order of magnitude.
- Built a multi-tenant retrieval platform with collection-level tenant isolation and async document ingestion, sustaining thousands of daily queries at sub-second p95 with zero cross-tenant leakage.
- Fine-tuned a detection model on a client dataset of over 100,000 annotated images, lifting mean average precision through systematic anchor, IoU threshold, and augmentation experiments.
- Built the team's offline evaluation suite and wired it into CI as a merge gate, so prompt and model changes stop shipping on intuition.
- Migrated inference from a hosted API to self-hosted vLLM, publishing a tokens-per-second, time-to-first-token, and quality-delta comparison that justified the switch.
- Mentor one junior engineer, review their pull requests weekly, and wrote the runbook and cost model the team uses when scoping a new AI feature.

PROJECTS

Six projects across four quarters, with a choice of capstone at the end. Each one is a whole system, so the skills compound instead of scattering. Quantify every result, because the seniors you are measuring against quantify everything.

Measurement changes character as the year goes on: correctness and throughput in Q1, judged output quality in Q2, cost and quality tradeoff curves in Q3, business outcomes in Q4. Each kind of evaluation is built where it first becomes meaningful, not before.

Q1 A model you wrote, served in production Oct – Dec 2026

Build a language model, then serve it yourself

PyTorch · pytest · FastAPI · Docker · GitHub Actions · OpenTelemetry · Grafana · no pre-built modelling code

- Implement BPE tokenization, embedding lookup, rotary positional encoding, grouped-query attention, KV cache, RMSNorm, and SwiGLU, then train a roughly 30M-parameter decoder to coherent generation on a small corpus.
- Prove the implementation is correct before trusting the loss curve. Load your weights into a reference implementation and assert your attention output matches within tolerance, then unit-test the causal mask, the axis the rotary embedding is applied on, and cached decoding against full recomputation. A flipped mask or an off-by-one in the cache still trains to a perfectly plausible loss, and this is the only check that catches it.
- Track held-out perplexity and a fixed set of qualitative prompts at every checkpoint, so a regression is visible during the run rather than discovered after it.
- Account for every byte of training memory before optimizing it. Work out on paper what parameters, gradients, optimizer states, and activations each cost at your batch size, then check the prediction against the allocator. Most people tune blind because they never did this once.
- Then actually optimize it: mixed precision, gradient accumulation, and gradient checkpointing, measuring the memory saved and the throughput given up for each. Profile a naive attention kernel against FlashAttention and write up where the activation memory goes.
- Do the same accounting for inference. KV-cache size is sequence length times layers times heads times head dimension times two, and knowing that number is what tells you how many concurrent users fit on one card. Measure it against what the allocator reports.
- Put your own weights behind a typed streaming API with request validation, structured logging, health checks, and a tested rate limiter, containerized and deployed through CI.
- Trace each request across tokenize, prefill, and decode with token counts attached, and put tokens per second and time to first token on the dashboard next to p95 latency.
- Learn what temperature and top-p actually do, then learn the thing people get wrong: temperature zero is not determinism. Batching, floating-point non-determinism, and silent provider updates all shift output, so never write a test that asserts an exact string. Make structured output reliable by using a schema or function calling rather than asking politely, validating with a typed model, retrying once with the validation error appended, capping retries, and logging the schema failure rate as a health metric. Teams lose weeks to a model that returns parseable output only ninety-six percent of the time.
- Load-test it, publish a latency budget, and hold p95 under it. The KV cache you wrote in the first half is the reason the second half is fast, so measure throughput with it on and off.

No golden set and no LLM judge here, deliberately. A 30M model on a small corpus has no task with a right answer, so a judge would score noise. Quality evaluation starts in Q2, where there is something to be right about.

Multi-tenant RAG platform, instrumented from day one

Qdrant · BM25 · cross-encoder reranker · RabbitMQ · LangFuse · OpenTelemetry · RAGAS

PHASE A · THE RETRIEVAL PIPELINE

- Build fixed, recursive, semantic, and contextual chunking paths plus hybrid sparse-dense retrieval and reranking behind one switchable config, then publish recall, precision, and faithfulness for every configuration on a hand-labelled question set. The ablation table is the deliverable, not the pipeline.
- Put a confidence interval on every row of that table. Work out the sample size you need before labelling, bootstrap the differences between configurations, and refuse to claim a winner when the intervals overlap. Most published retrieval comparisons are forty examples of noise presented as a result.
- Add metadata filtering, and learn the distinction that bites people: filtering before retrieval is usually correct, while filtering after it leaves you with two results when you asked for ten, because the filter threw the rest away.
- Implement small-to-big retrieval, indexing precise child chunks but handing the model the parent section, so search stays sharp without starving the answer of context. Add query rewriting that folds conversation history into a standalone question, because in any chat interface the raw follow-up retrieves nothing.
- Retrieve wide and narrow down rather than choosing between recall and precision: pull fifty candidates cheaply, rerank, keep five. Then look at your actual similarity score histogram before setting any threshold, because scores are only meaningful relative to your corpus and a hardcoded cutoff silently refuses to answer.
- Require a citation per claim and write a validator that rejects the answer when a cited id was not actually retrieved. It costs nothing and it eliminates invented sources. Then confirm the system declines a question your documents cannot answer instead of stretching to fill the gap.
- **Learn to localize a failure rather than guess at it.** Read context relevance, groundedness, and answer relevance together as a set. Low context relevance and low recall means the bug is in retrieval and no amount of prompt tuning will fix it. Good context with low groundedness means the model is ignoring what it was given. Being the person who says "recall at five is 0.62, this is a retrieval problem" is worth more than any single technique here.

PHASE B · THE PLATFORM AROUND IT

- Add per-tenant collection isolation and an async ingestion queue that decouples upload from embedding, then prove sub-second query latency and zero cross-tenant leakage under concurrent load.
- Go one step past tenant isolation to per-user and per-role access control on individual documents, enforced as a pre-filter from a server-side identity rather than anything the client sends. The leak in real systems is not a missing filter, it is permissions that went stale after someone changed roles, so sync them and write an adversarial regression case that tries to read what it should not.
- **Build the update and delete paths, not just ingestion.** Incremental upsert keyed on document id and content hash so reindexing is idempotent, deletion that propagates through the index and every cache rather than leaving orphans, and freshness metadata you can monitor. Then handle the migration everyone gets burned by once: changing the embedding model invalidates the entire index, so build the parallel-index cutover before you need it.
- Add a response cache in two layers, exact-match and semantic, and measure the false-hit rate. A loose similarity threshold serves a confidently wrong cached answer, and an unscoped cache hands one tenant another tenant's data, so scope the key by tenant and treat the threshold as something you tune against labelled data.
- Trace every request end to end with token counts and cost attached, carrying forward the telemetry you built in Q1.

PHASE C · THE OFFLINE EVALUATION SUITE

Build this generically, because the agent project adopts it next without modification. It is infrastructure, not a feature of this pipeline. Offline means it replays fixed cases without touching production, which is what makes it fast enough to gate a merge.

- **Make runs deterministic before anything else.** Pin seeds, drop temperature to zero where the task allows, and record every retrieval result and tool response as a fixture so a replay exercises the component you changed rather than the internet. A suite that gives different answers on identical input tells you nothing.
- **Score programmatically first, judge only where you must.** Exact match, schema validation, numeric tolerance, and assertions about structure are cheap, deterministic, and unarguable. Reach for an LLM judge only for the genuinely open-ended cases.

- **Design the rubric, because that is the actual skill.** Three to five ordinal points with every point defined in words, not a one-to-ten scale that produces noise you will spend a week chasing. Make the judge reason before it scores, give it two or three anchor examples per point, and constrain it to structured output so results are parseable and auditable.
- **Calibrate the judge against humans and publish the number.** Hand-label a few hundred outputs sampled across easy, hard, adversarial, and deliberately broken cases, then report weighted Cohen's kappa. Label twenty of them twice on different days first, because your own self-agreement is the ceiling: the judge cannot be more consistent than the humans defining correct. When you read the largest disagreements you will usually find the rubric was ambiguous rather than the model wrong, which is the most useful lesson in the project.
- **Quantify the judge's biases rather than just naming them.** Position bias, by running pairwise comparisons in both orders and measuring the flip rate. Length bias, by holding quality constant and correlating score against output length. Self-preference, by having one model family judge its own outputs against a competitor's and then swapping the judge. Prefer pairwise comparison for ship decisions and absolute scoring for regression thresholds. Almost no working engineer has measured any of these three.
- **Turn every production failure into a permanent case.** Mine your traces for failures, reduce each to a minimal reproducible case with the expected output, and add it to the corpus. This is the loop that makes the suite compound: it grows sharper exactly where your system is weak, and a bug fixed once stays fixed.
- **Report by slice, never by average alone.** An overall score hides a regression on one document type that a gain elsewhere cancels out. Break results down by tenant, query type, document length, and language, and treat any slice going backwards as a failure even when the headline number improved.
- **Diff runs, do not just score them.** The useful output when comparing two prompts or two models is the list of cases whose verdict changed, in both directions. Aggregate deltas tell you something moved; the diff tells you what, and lets you judge whether the trade was worth it.
- **Handle stochasticity honestly in the gate.** Model output varies even at temperature zero, so decide up front how you separate a real regression from noise: run the borderline cases several times, set thresholds from observed variance rather than round numbers, and quarantine genuinely flaky cases instead of letting them erode trust in the whole suite.
- **Put cost and latency budgets in the same gate as quality.** A change that lifts accuracy one point while doubling cost per request should fail the build, and it will not unless the suite is allowed to say so.
- Wire it into CI as a merge gate, so prompt and model changes stop shipping on intuition. Then make it runnable locally in under a minute on a subset, because a suite nobody runs before pushing is a suite that only ever delivers bad news.

Multi-agent system over MCP

LangGraph · Model Context Protocol · Postgres checkpointing · CrewAI

- Planner, executor, and critic agents sharing a tool registry exposed over MCP, with durable state so a run survives a process restart.
- **Write the harness, do not just wire a framework.** The loop that dispatches tools, parses model output, recovers from a malformed call, retries with backoff, enforces a step budget, and decides when to stop is the actual engineering here. Build it once yourself so you know what the framework is doing on your behalf.
- **Treat context as a budget you spend, not a box you fill.** Decide explicitly what earns a place in the window on each turn: system instructions, tool schemas, retrieved evidence, prior turns, scratchpad state. Measure how many tokens each category consumes in practice, because the answer is almost never what you assumed.
- Implement compaction properly. Summarize completed sub-tasks into structured state rather than prose, evict tool output that no longer informs the next decision, and keep the invariant that a compacted run can still answer questions about what it already did. Then measure where quality degrades as sessions grow long, which is the number that matters.
- Order the prompt so the stable prefix sits in front, and verify prompt caching is actually hitting rather than assuming it. A reordered system block silently doubles cost.
- **Design the tools, do not just register them.** Keep the set small and non-overlapping, because thirty tools with fuzzy boundaries produce fuzzy selection and no prompt tuning fixes that. Write descriptions that say when *not* to use each tool, use enums instead of free text, validate arguments before execution and hand back a structured error the model can correct rather than a stack trace, retrieve a relevant subset when the catalog is large, and always offer a none-of-these option so the model is never forced to pick the least bad tool.
- **Detect and stop loops, because this is the most common real agent failure.** Hash the tool name plus normalized arguments on every call, and when the hash repeats, block it and return the earlier result with a note that it was already tried, which forces a different branch. Track progress separately and terminate when several steps pass with no new information entering state. Enforce max steps, max tokens, max wall clock, and max cost per run in the orchestrator, never by asking the model in a prompt to be efficient.
- Name the four kinds of memory and treat them differently: working memory is the live context, episodic is what happened in past runs, semantic is durable facts about the user and domain, procedural is the routines encoded in your prompts and tools. Write to long-term memory through an explicit extraction step with a schema, never by dumping transcripts, because a raw dump carries speculation and corrections that get retrieved later as fact. Every record carries a source, a timestamp, and a confidence; contradictions resolve newest first with the old kept for audit; and there is an expiry policy and a size budget so memory cannot grow without bound.
- Gate human review on irreversibility and blast radius, not on model confidence. Money movement, data deletion, outbound messages, and anything a customer sees need approval. Show the reviewer a diff of the proposed action rather than a wall of reasoning, and feed approvals and rejections back into the eval set so the gate narrows over time. Add idempotency keys so a resumed run cannot execute a side effect twice.
- **Evaluate the trajectory, not only the outcome.** An agent can reach the right answer down a broken and expensive path. Measure tool selection accuracy, argument validity, step efficiency against the optimal path, and error recovery. When a task fails, open the trace and find the *first* wrong step, because everything after it is fallout. Watch for the failure that outcome scoring can never catch: silent success, where the agent skips a required step and returns a confident answer.
- Keep a named failure taxonomy rather than a list of individual bugs. Classify what goes wrong into recurring categories such as bad tool choice, bad arguments, lost context, or propagated upstream error, and fix classes rather than instances.
- Add a retry and escalation policy that distinguishes transient failures from terminal ones, passing terminal ones back to the model as facts about the world. Fan out independent tool calls in parallel with a concurrency cap, which is usually the largest single latency win available in an agent.
- Adopt the harness from the previous project without modifying it, and report task completion rate, tool-call accuracy, and steps to completion. If the agent work forces a change to the harness, it was written for one consumer and needs rewriting.
- Rebuild one workflow in a second framework to find out what the abstraction was hiding.

One system that forces training, distillation, serving, and speech together

Unsloth · TRL · PEFT · vLLM · AWQ and GGUF · DeBERTa · Whisper · Silero VAD · streaming TTS · LangGraph

Voice sets a hard latency budget. The budget is what makes quantization, self-hosted serving, and a distilled router necessary rather than optional, which is why these belong in one project instead of four.

Be honest about scope. This is the heaviest quarter in the year and the phases are not equally sized. If you are short on time or compute, the rung to cut is the reward model and PPO loop, which is the most expensive and the least used in practice. Read it, understand reward hacking, and skip straight from preference optimization to a verifier. Cutting Phase A is not an option, because a bad corpus caps everything above it.

PHASE A · THE DATASET, WHICH IS MOST OF THE WORK

- Build the banking dialogue corpus rather than assuming one exists. Source it, deduplicate near-identical turns, filter for quality, and analyse what you actually have: turn-length distribution, intent balance, vocabulary coverage, and how many examples cover each transaction type. An unbalanced corpus produces a model that is confident about balance enquiries and lost on transfers.
- Check the training set for contamination against your evaluation set before training anything, because a leaked overlap will show up as a benchmark win you cannot reproduce in production.
- Label a slice twice, measure inter-annotator agreement, and fix the labelling guidelines where annotators disagree. The ceiling on model quality is the consistency of the labels, so measure the ceiling first.
- Write the dataset card: sources, cleaning decisions, known gaps, and licence. This is the artifact that makes the training runs reproducible.

PHASE B · THE MODEL LAYER, HEADLESS AND BENCHMARKED

- **Choose the base model deliberately and write down why.** Shortlist three or four candidates across sizes and licences, evaluate them on your own task before any training, and compare them on quality, licence terms, context length, tokenizer fit for your domain vocabulary, and what they cost to serve at your expected load. Run this through the offline suite from Q2, which is exactly the kind of question it was built to answer. Picking the model everyone is posting about and discovering the licence forbids your use case is a mistake you only make once.
- Continued pre-training on banking and financial text, then supervised fine-tuning with loss masking and sequence packing on transaction dialogue. Use LoRA and QLoRA, and understand why quantizing the frozen base is what makes this fit on the card you can afford.
- Mix general data into the domain corpus to measure and mitigate catastrophic forgetting, and report the benchmark delta at every rung. Publish the adapter with a model card.
- Make every run reproducible from its config alone. Track experiments properly, log loss curves and validation metrics and sample outputs per checkpoint, and run a real sweep over at least rank and learning rate so you have a trade-off to discuss rather than one lucky run. Record why you chose the checkpoint you shipped.
- Distill the intent routing hop out of the hot path: use the large model to label routing data, train a small encoder classifier to take the decision, and hold accuracy within one point at a fraction of the cost and latency. Document exactly where the small model fails.

PHASE C · ALIGNMENT, CLIMBING ALL THREE RUNGS

- **Direct preference optimization first**, because it is the cheap rung. Build the preference pairs, train, and note what it can and cannot express. Be clear with yourself that this is not reinforcement learning: there is no sampling loop and no reward model, which is exactly why it is where you start.
- **Then classic RLHF once, properly.** Train a reward model on human preference data, run proximal policy optimization against it with a KL penalty back to the frozen reference, and tune that penalty until you understand what it is buying you. Do this rung not for the algorithm but to watch reward hacking happen: let the KL term go slack on purpose and read what the model produces when it learns to satisfy the proxy instead of the intent. You will recognize that failure everywhere afterwards.
- **Then GRPO against a verifier you wrote**, which is where useful work actually is now. No reward model, group-relative advantage, and rewards that come from checking the answer rather than judging it.

- **The environment is the skill, not the trainer.** Anyone can run the training loop. Design the environment: what an episode is, what the model observes, which actions are legal, and what terminates a run. Then write the verifier, meaning the program that decides whether a rollout succeeded, such as whether the extracted transaction parameters match ground truth, the amount parsed correctly, and the recipient resolved to a real account.
- **Then attack your own reward.** Write down three ways the model could score well while doing the wrong thing, check whether it found any of them, and fix the reward rather than the model. A verifier that cannot be gamed is a harder artifact to build than the policy that trains against it.
- Report the win rate of each rung against the last, and be honest when a rung bought you nothing.

PHASE D • SERVING ECONOMICS

- Self-host with paged attention and continuous batching, then sweep AWQ, GPTQ, and GGUF across batch sizes. Publish tokens per second, time to first token, VRAM use, and quality delta, with the concurrency curve where throughput stops improving.
- Tie it back to the memory accounting you did in Q1. Paged attention exists because KV-cache memory fragments, and quantization exists because weights dominate until the cache does. You should be able to predict the VRAM number before you measure it.
- Ship this phase as a text-in, text-out API with its own benchmark report, so there is a working artifact before voice touches it.

PHASE E • THE VOICE LAYER ON TOP

- Add voice activity detection, streaming ASR, partial-hypothesis routing, barge-in, and sentence-level TTS streaming for a full-duplex loop.
- Drive end-to-end turn latency under 800ms and instrument every stage, measuring the voice layer against the text baseline you already trust so you know which stage regressed.
- Enforce transaction safety: confirmation before any irreversible action, slot-filling with explicit read-back of amounts and recipients, and a refusal path when confidence is low.

The operating layer

LiteLLM · FastAPI · Redis · Prometheus · Grafana · garak · feature flags

Why this exists. Everything up to here taught you to build. This teaches you to run, which is the half that separates an engineer who ships a demo from one a company trusts with production. It needs no new domain: it points at the systems you already built in projects one through five, so the cost is infrastructure work rather than research.

PHASE A · A GATEWAY THAT SURVIVES ITS PROVIDERS

- Route every model call in your projects through one service that normalizes providers, and require tenant, feature, and request id on every call. Without that metadata you can never attribute cost, and attribution is half the reason gateways exist.
- Track per-provider health in a rolling window: success rate, p50, p95, p99, and an error taxonomy that separates rate limits, timeouts, server errors, content filters, and auth failures. Instrument before you build failover, because failover you cannot observe is guesswork.
- Add a circuit breaker per provider with closed, open, and half-open states. The half-open probe is what makes it self-healing: send a trickle back to a recovering provider, close on success, reopen immediately on failure.
- Fail over by request class, since a cheap classification call and a long generation should not be treated alike. Add idempotency keys so a retry cannot double-charge, and queue deferrable work with backoff and jitter instead of returning an error.
- Then break it on purpose. Inject errors and latency into one provider and watch the breaker trip, traffic reroute, and the breaker close again on the dashboard. Failover you have not tested is a hypothesis.

PHASE B · A RELEASE PATH, NOT A DEPLOY BUTTON

- Stage every change through the same ladder: the offline suite gates it, shadow mode runs it on real traffic without showing anyone the output, a canary takes one to five percent with automatic rollback triggers, then an A/B test with the primary metric and sample size registered before you look, then full rollout with the old version behind a flag for a cycle.
- Never change the prompt and the model in the same deploy. You will not be able to attribute the result, and you will argue about it for a week.
- Version everything that changes behaviour, not just application code: prompts, rubrics, tool schemas, evaluation sets, and the retrieval index. An index rebuilt quietly is a deploy nobody recorded.

PHASE C · EVALUATION THAT RUNS IN PRODUCTION

- Offline evaluation proved the change was good before shipping. Online evaluation proves it is still good on traffic your eval set never imagined. Sample live outputs through a calibrated judge for groundedness and policy adherence rather than scoring everything, and pin the judge version or your scores stop being comparable week to week.
- Mine implicit signals, which are free and high volume: did the user edit the output, copy it, abandon it, or immediately rephrase the question. A rephrase is one of the strongest proxies you have for a failed first answer.
- Watch three kinds of drift: input drift as users start asking new things, quality drift as online scores slide, and embedding drift as the corpus shifts. Alert on metric regressions, not only on exceptions, because the dangerous failures never throw.
- Practise the diagnosis, because the instinct to retrain first is wrong. Work the dependency chain: did the index go stale or half-update, did an upstream parser change, was a prompt or model version deployed, did a new traffic segment arrive speaking differently, did the provider change a default. Compare the failing cohort against a stable one before you touch the model.
- Decide what pages someone and what gets reviewed weekly. Cost, latency, and error rate page. Quality trends are read on a schedule.

PHASE D · ATTACK YOUR OWN SYSTEMS, ON A SCHEDULE

- Build an attack battery covering direct jailbreaks, system-prompt extraction, PII extraction, tool abuse and privilege escalation, encoding tricks, and above all indirect injection: plant a poisoned document in your own retrieval corpus and find out whether the pipeline follows its instructions. That is the attack most real systems are actually exposed to, and the one people forget.

- Point it only at applications you own and control, namely your own earlier projects. Never at third-party systems.
- Score with deterministic checks first, since they are free and unambiguous: did the system prompt leak verbatim, did a PII pattern appear, did a blocked tool fire. Use three outcomes rather than two, because the needs-review queue is where you learn what your scorer misses.
- Run it scheduled and on every deploy. A confirmed finding becomes a permanent regression case that is never removed once fixed, and the dashboard tracks pass rate by category with a regression count that must stay at zero.

PHASE E • UNIT ECONOMICS

- Measure before optimizing, by stage. Cost usually concentrates in input tokens rather than output, and latency in retrieval plus one oversized generation, but assume neither until you have the numbers.
- Cache in three layers: exact-match responses, a semantic cache on normalized query embeddings with a strict threshold and a TTL, and provider prompt caching on the static prefix.
- Shrink the input, tier the models so a small one handles the easy majority and escalates on low confidence, and track the escalation rate as the tuning dial it is.
- Finish with the number that reads most senior in the whole year: the break-even request volume above which self-hosting beats the API. Report cost per thousand requests and p95 before and after, with the quality delta from your eval set beside them, because a saving quoted without a quality number reads as a red flag.

Then the capstone. Two options, and you build one. They are deliberately different in shape: the first goes deep on vision and document systems for a paying business problem, the second goes wide and picks up the speech, language adaptation, and serving economics the first one never touches. Pick by which gaps you most need to close, using the coverage table at the end of this section.

Freight document reconciliation

YOLO · RT-DETR · PaddleOCR · spaCy NER · Qdrant · LangGraph · Kubernetes · Terraform

The business. A logistics operator moves *loads*. One load is one shipment: goods collected at A, delivered to B, by one truck, invoiced to a customer. The company moves thousands a month.

The paperwork. Every load generates physical documents that travel in the truck cab. A bill of lading, the contract stating what goods and how many. A proof of delivery, signed at the destination dock, recording the quantity actually received and the times the truck arrived and left. A weighbridge ticket. Fuel and toll receipts.

Where the money leaks. A load cannot be invoiced until its proof of delivery is verified, and that document is a piece of paper in a truck. The driver photographs it at the dock, in bad light, at an angle, often a faint carbon copy with handwriting on it. Back-office staff read the photo and key the numbers in by hand. Billing therefore lags delivery by days, and the gap is working capital sitting idle. When the bill of lading says 500 cartons and the delivery slip says 487, someone investigates by scrolling through photos and phoning the driver. When a customer disputes detention time, the evidence is a timestamp buried in an image.

The system. The driver photographs the document. Within seconds the load is either marked billable with no human involved, or it reaches one person's desk with the discrepancy already found and explained.

PHASE A · READ THE DOCUMENT

- Annotate a custom dataset, fine-tune a detector to locate the document in a cluttered photo and classify which type it is, and lift mean average precision through deliberate experiments on anchor calibration, IoU thresholds, non-max suppression, and mosaic and HSV augmentation. Report the ablation, not just the final number. Real driver photos vary enormously, which is exactly why this tuning matters.
- Deskew from the detected boundary, crop, and apply adaptive thresholding to recover faint carbon copies and shadowed corners, then run OCR over the corrected regions including handwritten quantities and signatures.
- Parse the OCR text through a spaCy NER model with a custom schema covering consignment number, quantity received, date, time in, time out, weighbridge reading, and signature presence. Link every field back to the pixel region it came from, so a reviewer sees the crop rather than trusting a number.

PHASE B · MATCH AND RECONCILE

- Match the extracted consignment to a load using fuzzy string matching combined with semantic matching on date, route, customer, and quantity. A handwritten 3 reads as an 8 and a 1 reads as a 7, so the exact-match path fails often enough that it cannot be the only path.
- Compare extracted values against what the system expected. When quantity agrees, a signature is present, and delivery falls inside the window, mark the load billable and stop. No human touches it.
- When something disagrees, an agent pulls the original booking, the customer's history of short deliveries, and the driver notes, then either clears the case under an explicit policy or writes a case file a human reads in thirty seconds instead of investigating for twenty minutes. Use your own fine-tuned small model for those summaries, because per-document API cost compounds at this volume.

PHASE C · THE OPERATIONAL DATA IT ACCUMULATES

- After a few months this system holds a warehouse: every document, extraction confidence, mismatch, escalation, and resolution. Model the schema properly and build the dashboards that answer the standing questions, such as auto-clear rate by document type and turnaround by lane.
- Add a natural-language query layer over that schema, so an operations manager can ask which customers dispute most this quarter without writing SQL. Ground the generated queries in the real schema, constrain them to read-only, and

validate before execution.

- Use the accumulated data to tune the system itself: find where extraction confidence predicts an actual error, and set the auto-clear threshold from that curve rather than from a guess.

PHASE D · RUN IT LIKE IT BELONGS TO YOU

- Production deployment with SLOs on time to billable, PII redaction on driver and consignee data, rate limits, an on-call runbook, and a cost model that predicts the monthly bill within ten percent.
- Report the numbers that decide whether it stays: share of documents auto-cleared without a human touch, median hours from photo to billable, cost per document, human review minutes per escalation, and the disputed-load rate before and after.

A voice-first crop advisor for smallholder farmers

Whisper fine-tuning · regional-language TTS · YOLO · hybrid RAG · CPT and SFT and GRPO · vLLM with AWQ
· LangGraph · WhatsApp Business API

The problem. Pakistan has millions of smallholder farms, most of them a few acres, and a handful of government extension officers for every thousand farms. A farmer sees yellowing on his wheat. Is it rust, nitrogen deficiency, or waterlogging? Guess wrong and he either sprays a fungicide he cannot afford for a problem he does not have, or loses the crop. The correct answer exists, in provincial extension bulletins and pesticide labels, but those are English PDFs written for agronomists. The farmer may not read, and he speaks Punjabi, Sindhi, or Pashto rather than the Urdu most tools assume.

The system. The farmer sends a WhatsApp voice note in his own language and a photo of the affected plant. He gets back a spoken answer in that language: what it probably is, what to do, what quantity, what it will cost, and when to act. If the system is not confident, it says so and routes him to a human extension officer instead of guessing.

PHASE A · HEAR HIM

- Collect and label real audio. Off-the-shelf recognisers are weak on Punjabi and Sindhi and worse on code-switched rural speech recorded outdoors on a cheap handset. There is no dataset waiting for you, so building one is the first task: recording protocol, transcription guidelines, a second pass for agreement, and a held-out test set per language.
- Fine-tune the recogniser per language and report word error rate for each, separately, because an average across languages hides the one that fails. This is the part of the project that cannot be bought.
- Handle the practical shape of the input: voice notes arrive as whole files rather than a live stream, run thirty seconds or longer, contain background noise from machinery and animals, and often start mid-sentence.

PHASE B · UNDERSTAND WHAT HE IS ASKING

- Adapt the language model to the domain. This is the rare case where continued pre-training is genuinely necessary rather than decorative, because the base model has thin coverage of regional-language agricultural vocabulary, local crop names, and the names farmers actually use for pests. Then supervised fine-tuning on advisory dialogue.
- Run an agent that fills the slots a real agronomist would ask for before answering: which crop, what growth stage, when symptoms appeared, how far they have spread, what was last applied to the field. Ask over voice, one question at a time, and carry the answers across a conversation that may pause for hours.
- Manage context carefully here. The window holds the retrieved bulletins, the slot state, the conversation so far, and the image findings, and the conversation is the part that grows. This is the Q2 context-engineering work under a real constraint.

PHASE C · LOOK AT THE PLANT

- Classify leaf and stem disease from a phone photo taken in field conditions: variable light, cluttered background, partial leaves, a thumb in frame. Fine-tune on a plant disease dataset and then on images you collect locally, because the public datasets are mostly lab photographs on clean backgrounds and will not survive contact with a real field.
- Make abstention a first-class output. A confident wrong diagnosis costs a crop, so the model must decline when it is outside what it knows. Calibrate the confidence, pick the threshold from the confidence-versus-error curve, and measure abstention accuracy, meaning how often it declined in cases where it would have been wrong.
- Fuse the image finding with the spoken symptoms rather than trusting either alone. Agreement between the two raises confidence; disagreement is a reason to ask one more question or escalate.

PHASE D · ANSWER SAFELY

- Ground every answer in retrieval over extension bulletins, registered product labels, and provincial crop calendars, with hybrid search and reranking. A dosage figure with no retrieved source is refused, not improvised, because pesticide dosage is both a poisoning risk and a legal limit.

- Multi-tenancy by province, crop, and season, since each selects a different advisory corpus and a different calendar. One deployment, many isolated knowledge bases.
- Hard guardrails with a real refusal path. Restricted chemicals always escalate to a human extension officer. Low confidence escalates. Anything involving pre-harvest intervals or livestock exposure escalates. The system must be willing to say it does not know.
- Train the final rung with GRPO against a verifier that checks whether the advice is safe and correct: dosage within the label range, the product registered for that crop, the timing consistent with the growth stage. Then try to game your own reward before the model does.
- Speak the answer back in the farmer's language, with the action, the quantity, the cost, and the timing, in that order.

PHASE E · MAKE IT AFFORDABLE, THEN LEARN FROM IT

- Cost per interaction must be near zero for this to reach anyone, which forces a quantized self-hosted model and makes the Q3 benchmark the thing that decides the architecture rather than a report nobody reads.
- The queries cluster by district and by date, which makes the system an early-warning signal for disease outbreaks. That dashboard is plausibly worth more to an agriculture department than the individual advice is.
- Feed real outcomes back: which advice was followed, what happened to the crop, where the model was wrong. This is the flywheel, and most projects never build it.

WHAT YOU MEASURE

- Agreement with a qualified agronomist on a held-out set of real queries. This is the only quality number that matters, and everything else is a proxy for it.
- Word error rate per language, share of queries resolved without escalation, abstention accuracy, and cost per interaction.
- Run a pilot with a real cohort of farmers and report what changed for them. A capstone nobody uses is a demo.

The crop advisor covers more of the year and is the better choice if the goal is breadth. The freight system is the better choice if you want depth in vision and document pipelines, or if you want a capstone that maps onto a business that will pay for it. Neither is wrong. Building both in one quarter is. The matrix below shows exactly what each one closes.

SKILL MATRIX

Every skill in the year, in the order you first meet it, against the project that introduces it and every project that uses it again. Read down a column to see what one project demands. Read across a row to see how long a skill stays with you. The columns get denser as the year goes on, which is the point: nothing is learned once and abandoned.

N introduced here · **D** pushed further than before · **R** applied again as it stands · · not used

How to read a row. First is always the leftmost mark, and nothing appears to its left, because you cannot reuse something you have not learned yet. Every row runs left to right and never doubles back. A skill you touch lightly early and study properly later reads First then Deeper, which is the common case: you write a KV cache in project 1 and only optimize it against paged attention in project 4. The single exception is the two capstones, which are alternatives rather than a sequence, so a skill may be introduced independently in either branch.

SKILL	1 · MODEL & SERVING	2 · RAG PLATFORM	3 · AGENT SYSTEM	4 · VOICE BANKING	5 · OPERATING LAYER	6A · FREIGHT DOCUMENTS	6B · CROP ADVISOR
Q1 · FOUNDATIONS, INTRODUCED BUILDING THE MODEL							
Transformer internals	N	·	·	R	R	·	R
Tokenization	N	R	R	D	R	·	D
Training loop & mixed precision	N	·	·	D	R	R	D
Training-memory accounting	N	·	·	D	R	R	D
KV cache & inference memory	N	·	·	D	R	·	D
Numerical correctness testing	N	R	R	R	R	R	R
Structured-output reliability	N	·	R	D	R	R	R
Service & streaming API design	N	R	R	D	R	R	D

SKILL	1 • MODEL & SERVING	2 • RAG PLATFORM	3 • AGENT SYSTEM	4 • VOICE BANKING	5 • OPERATING LAYER	6A • FREIGHT DOCUMENTS	6B • CROP ADVISOR
Containerization & CI/CD	N	R	R	R	D	D	D
Tracing & telemetry	N	D	D	D	D	R	R
Load testing & latency budgets	N	R	R	D	D	R	D
Operational dashboards	N	R	R	D	D	D	D
SLOs, alerting & on-call	N	.	.	R	D	D	D
Q2 • RETRIEVAL AND MEASUREMENT							
Chunking strategy	.	N	R	R	R	.	R
Embeddings & vector search	.	N	R	R	R	D	R
Hybrid sparse-dense retrieval	.	N	R	R	R	D	D
Reranking	.	N	R	R	R	.	R
Retrieval metrics	.	N	R	R	R	R	R
Statistical significance	.	N	R	D	R	R	D
LLM-as-judge calibration	.	N	R	D	R	R	D
Eval harness & CI gating	.	N	D	D	D	R	R
Deterministic replay & fixtures	.	N	R	R	R	R	R
Programmatic scoring over judging	.	N	R	D	R	R	D
Regression corpus from traces	.	N	D	D	D	D	D
Slice reporting & run diffing	.	N	R	D	R	D	D
Flakiness control in gates	.	N	R	D	R	R	D
Cost attribution per request	.	N	R	D	D	D	D
Cost & latency budgets in CI	.	N	R	D	R	D	D
Multi-tenancy & isolation	.	N	.	R	R	.	D
Async ingestion & queues	.	N	R	.	R	D	R
Index lifecycle & migration	.	N	.	R	D	R	R
Document-level access control	.	N	.	.	D	R	R
Response & semantic caching	.	N	R	R	D	R	D
Dataset construction & curation	.	N	.	D	R	D	D
Annotation & agreement	.	N	.	D	R	D	D
Context engineering	.	N	D	D	R	R	D
Query rewriting & transformation	.	N	R	R	R	R	D
Q2 • AGENTS AND HARNESS							
Tool calling & MCP	.	.	N	R	R	R	R
Harness engineering	.	.	N	D	R	R	D
Planning patterns	.	.	N	R	R	R	R
Compaction & prompt caching	.	.	N	D	R	R	D
Agent memory & durable state	.	.	N	D	R	R	D
Human-in-the-loop & escalation	.	.	N	D	R	D	D
Agent evaluation	.	.	N	R	R	R	R
Tool design discipline	.	.	N	D	R	R	D
Loop detection & budget caps	.	.	N	D	D	R	D
Trajectory evaluation	.	.	N	D	D	R	D
Failure taxonomy	.	.	N	D	D	D	D
Guardrails, PII & injection defence	.	.	N	D	D	D	D
Q3 • TRAINING AND ALIGNMENT							
Base-model selection	.	.	.	N	R	D	D

SKILL	1 • MODEL & SERVING	2 • RAG PLATFORM	3 • AGENT SYSTEM	4 • VOICE BANKING	5 • OPERATING LAYER	6A • FREIGHT DOCUMENTS	6B • CROP ADVISOR
Continued pre-training	*	*	*	N	R	*	D
Supervised fine-tuning	*	*	*	N	R	R	D
LoRA & QLoRA	*	*	*	N	R	R	R
Preference optimization (DPO)	*	*	*	N	R	*	R
RLHF: reward model & PPO	*	*	*	N	R	*	*
RL environments & verifiers	*	*	*	N	R	*	D
Reward-hacking mitigation	*	*	*	N	R	*	D
Catastrophic-forgetting control	*	*	*	N	R	*	R
Knowledge distillation	*	*	*	N	R	R	R
Quantization	*	*	*	N	D	D	D
Self-hosted serving & batching	*	*	*	N	D	R	D
Inference benchmarking	*	*	*	N	R	R	D
Experiment tracking & sweeps	*	*	*	N	R	D	D
Calibrated uncertainty & abstention	*	*	*	N	R	D	D
Cost modelling	*	*	*	N	D	D	D
Q3 • SPEECH							
ASR, VAD & streaming	*	*	*	N	R	*	D
TTS & barge-in	*	*	*	N	R	*	R
Real-time latency engineering	*	*	*	N	R	*	R
Q4 • THE OPERATING LAYER							
Multi-provider failover	*	*	*	*	N	*	*
Circuit breaking & self-healing	*	*	*	*	N	*	*
Chaos testing	*	*	*	*	N	*	*
Shadow, canary & A/B rollout	*	*	*	*	N	*	*
Artifact versioning & rollback	*	*	*	*	N	*	*
Online evaluation on live traffic	*	*	*	*	N	*	*
Implicit feedback signals	*	*	*	*	N	*	*
Drift detection & diagnosis	*	*	*	*	N	*	*
Red-team harness in CI	*	*	*	*	N	*	*
Model tiering & escalation rate	*	*	*	*	N	*	*
Break-even unit economics	*	*	*	*	N	*	*
Q4 • INTRODUCED ONLY IN A CAPSTONE							
Detection & fine-tuning	*	*	*	*	*	N	N
Anchor, IoU, NMS & augmentation	*	*	*	*	*	N	N
OCR & image preprocessing	*	*	*	*	*	N	*
NER & information extraction	*	*	*	*	*	N	*
Fuzzy & semantic record matching	*	*	*	*	*	N	*
Schema modelling & SQL analytics	*	*	*	*	*	N	N
Natural language to SQL	*	*	*	*	*	N	*
Infrastructure as code	*	*	*	*	*	N	N
Multimodal fusion	*	*	*	*	*	*	N
ASR fine-tuning, low-resource	*	*	*	*	*	*	N

Three things are worth noticing. Tracing, correctness testing, and containerization appear in every column after the one that introduces them, which is what it looks like when a habit rather than a technique has been learned. The Q1 column is the only one that is all First marks, because at the start everything is new and nothing is yet being reused. And the operating-layer column is almost entirely Deeper

and Reused marks over skills introduced earlier, which is exactly what a project about running things rather than building them should look like.

HOW EACH PROJECT SHIPS

A project nobody can evaluate in two minutes is an unfinished project. Every one of these gets the same treatment, and it takes an afternoon.

- One line at the top of the README saying what it does and what it achieved, with the number in it. Then the result immediately: a benchmark table or a dashboard screenshot, before any architecture.
- A demo under two minutes. Most people reviewing your work will never run the code. They watch, read the top of the README, and decide.
- A short design-decisions section naming the two or three real trade-offs you made and what you gave up, and a section on what did not work. Experienced engineers read that section first, and almost nobody writes it.
- Real data. Public filings, your own documents, open datasets. Toy data produces toy results and it is obvious instantly.
- One sentence you can say out loud about the problem and the number, not the stack. Not the frameworks you used, but that runs were losing four minutes of work on every deploy so you made the state durable. Then have one specific failure ready per project, because that question always comes.

JUDGMENT, NOT JUST TECHNIQUE

- **Know when not to reach for a language model at all.** Gradient boosting still wins on tabular data, a regex beats a model on a fixed format, and a lookup table beats both. Make that call explicitly at least once during the year and write down the reasoning, because choosing the boring tool when it is correct is a senior signal and choosing a model for everything is a junior one.
- Carry the classical metric vocabulary even though you are not doing classical ML this year: precision against recall and which error is costlier, why accuracy lies on imbalanced data, what a precision-recall curve shows that a receiver operating characteristic hides, and how to tune a threshold rather than accepting the default. Your distilled router and your detector both need this.
- Know the four ways data leaks into an evaluation and check for each: preprocessing fitted before the split, temporal leakage from using the future, group leakage where the same entity spans train and test, and target leakage from a feature that encodes the answer.
- Least privilege plus deterministic gating is the answer to injection, exfiltration, agent guardrails, and access control alike. A model decides what to propose; code decides what is permitted to happen. If you internalize one principle from the whole year, make it that one.
- Abstention is a success state, not a failure. A system that reliably says it could not find something is worth more than one that is right slightly more often and occasionally fabricates. Track false refusals next to unsupported answers, false blocks next to violations, and false cache hits next to hit rate. Every threshold has two error directions and juniors only ever watch one.

OPEN SOURCE, RESEARCH & WRITING

- Three merged pull requests to a library you actually use, at least one a substantive fix rather than a documentation change.
- One paper reproduced from scratch, with a public write-up of where your numbers diverge from the authors'.
- Six technical write-ups, one every two months, each carrying a measurement nobody else has published.
- Mentor one person who started after you, and review their code weekly. Explaining a system is the fastest way to find out you do not understand it.

WHAT DONE LOOKS LIKE

QUARTER	SHIPS BY	PROOF IT IS REAL
Q1	31 Dec 2026	A transformer you wrote line by line matches a reference implementation within tolerance, and is generating readable text from behind a public endpoint under a latency budget you published.
Q2	31 Mar 2027	A stranger can pick a retrieval configuration from your ablation table. Your evaluation suite has already blocked one of your own merges and been right to. Your agent survives a restart mid-run, and it adopted the suite without changing it.
Q3	30 Jun 2027	You can hold a spoken conversation with a model you trained on a corpus you built, running on hardware you rented, and quote its cost per thousand calls against the API it replaced.
Q4	30 Sept 2027	You degraded a provider on purpose and traffic kept flowing. A canary rolled itself back without you. Your red-team suite found a real hole and now guards it permanently. Real users are putting real inputs into your capstone, you have been paged, and you fixed it.

EDUCATION & CERTIFICATIONS

Education

BS Computer Science — [University], Islamabad

Certifications

- Deep Learning Specialization — Coursera
- Machine Learning Engineering for Production — Coursera
- One cloud associate certification, Azure AI Engineer or AWS

Where the data work lives. There is no standalone data science project, because in this job the data work is never standalone. It is the corpus you build in Q3, which determines the ceiling on every training run above it. It is the significance testing in Q2, without which an ablation table is decoration. It is the warehouse the capstone accumulates and the questions you can then answer from it. Treating data as a separate project is how it ends up being nobody's job.

Deferred, not dropped. Classical predictive modelling, meaning gradient boosting, class-imbalance handling, and the tabular craft, is out of scope this year and belongs in year two. It matters, but it is not what makes an AI engineer, and the statistical literacy it teaches is picked up in Q2 and Q3 instead.

The point of this document is not to be printed. It is to be shortened. Every line you can honestly keep is a line you earned, and every line still aspirational tells you what to build next month.